

ALFIDOTECH INTERNSHIP

Task 2: Deep Learning Image Classification

Project: CIFAR-10 Image Classification using Transfer Learning

Author: C. Rajalakshmi

Framework: PyTorch | **Model:** Pretrained ResNet18 | **Device:** CUDA GPU

1. Project Overview

A deep learning image classifier was implemented using PyTorch and a pretrained ResNet18 model. The final classification layer was replaced for the 10 CIFAR-10 classes, while pretrained layers were frozen.

2. Dataset and Preprocessing

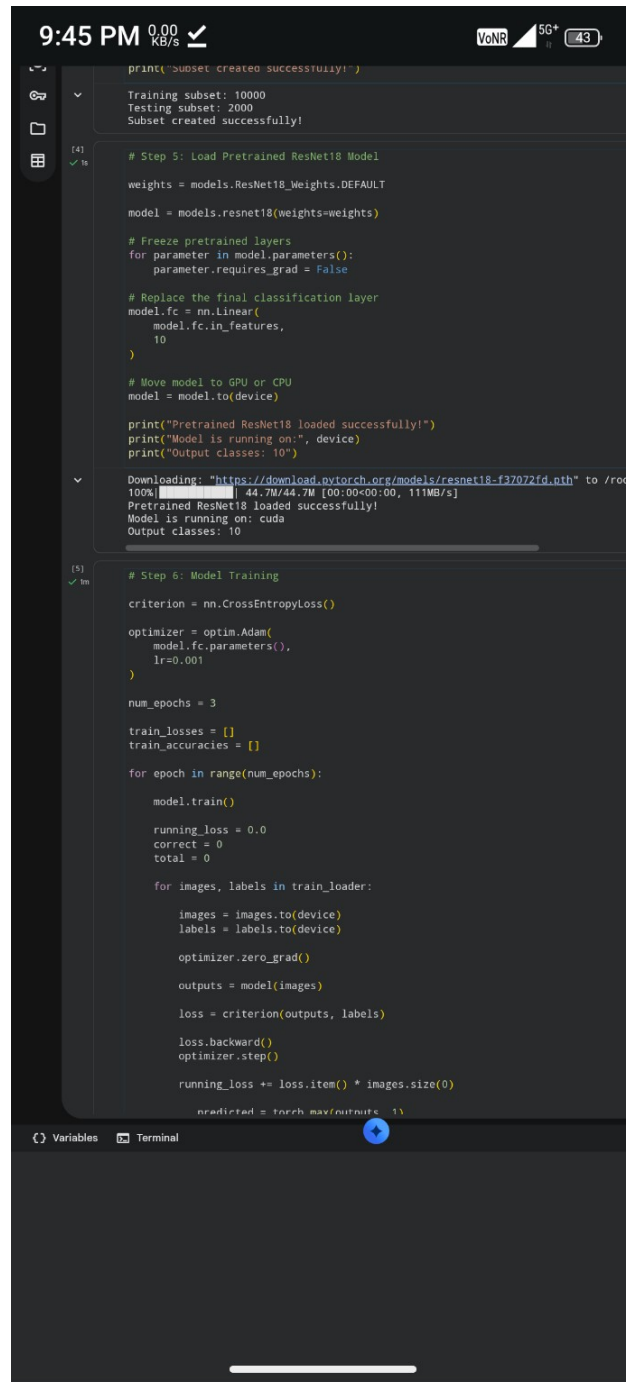
Item	Details
Dataset	CIFAR-10
Training subset	10,000 images
Testing subset	2,000 images
Classes	10
Augmentation	Random crop and horizontal flip
Preprocessing	Resize to 224 x 224 and ImageNet normalization

3. Training Results

Epoch	Loss	Training Accuracy
1	1.4189	56.04%
2	0.8791	72.91%
3	0.7805	74.68%

Test Accuracy: 76.90% on the 2,000-image test subset.

4. Training Curves



```
9:45 PM 0.00 KB/s VoNR 5G+ 43

print("Subset created successfully!")

Training subset: 10000
Testing subset: 2000
Subset created successfully!

[4] ✓ 1s
# Step 5: Load Pretrained ResNet18 Model

weights = models.ResNet18_Weights.DEFAULT
model = models.resnet18(weights=weights)

# Freeze pretrained layers
for parameter in model.parameters():
    parameter.requires_grad = False

# Replace the final classification layer
model.fc = nn.Linear(
    model.fc.in_features,
    10
)

# Move model to GPU or CPU
model = model.to(device)

print("Pretrained ResNet18 loaded successfully!")
print("Model is running on:", device)
print("Output classes: 10")

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root
100%|██████████| 44.7M/44.7M [00:00<00:00, 111MB/s]
Pretrained ResNet18 loaded successfully!
Model is running on: cuda
Output classes: 10

[5] ✓ 1m
# Step 6: Model Training

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.fc.parameters(),
    lr=0.001
)

num_epochs = 3

train_losses = []
train_accuracies = []

for epoch in range(num_epochs):

    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, labels)

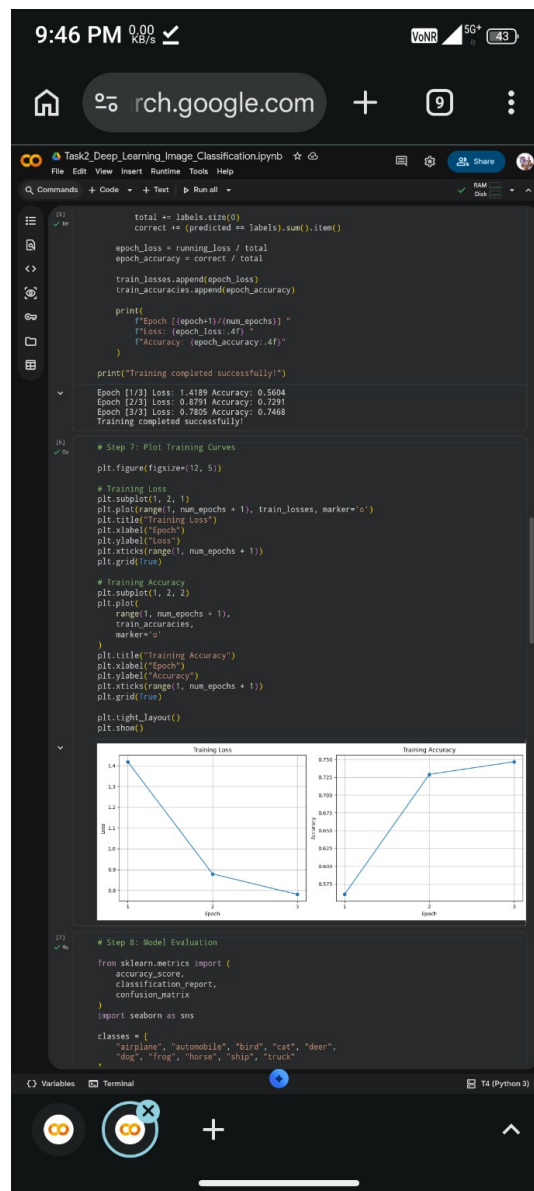
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    predicted = torch.max(outputs, 1)
```

5. Model Evaluation

Evaluation included test accuracy, class-wise precision, recall, F1-score, and a confusion matrix. The overall weighted average F1-score was approximately 0.77.



9:46 PM 0.00 KB/s

VoNR 5G+ 43



rch.google.com



```
Task2_Deep_Learning_Image_Classification.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk

# Step 8: Model Evaluation

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
import seaborn as sns

classes = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

model.eval()

all_predictions = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        predictions = torch.argmax(outputs, dim=1)
        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(labels.numpy())

# Test Accuracy
test_accuracy = accuracy_score(
    all_labels,
    all_predictions
)

print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Classification Report
print("\nClassification Report:\n")

print(
    classification_report(
        all_labels,
        all_predictions,
        target_names=classes,
        zero_division=0
    )
)

# Confusion Matrix
cm = confusion_matrix(
    all_labels,
    all_predictions
)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="e",
    cmap="Blues",
    xticklabels=classes,
    yticklabels=classes
)

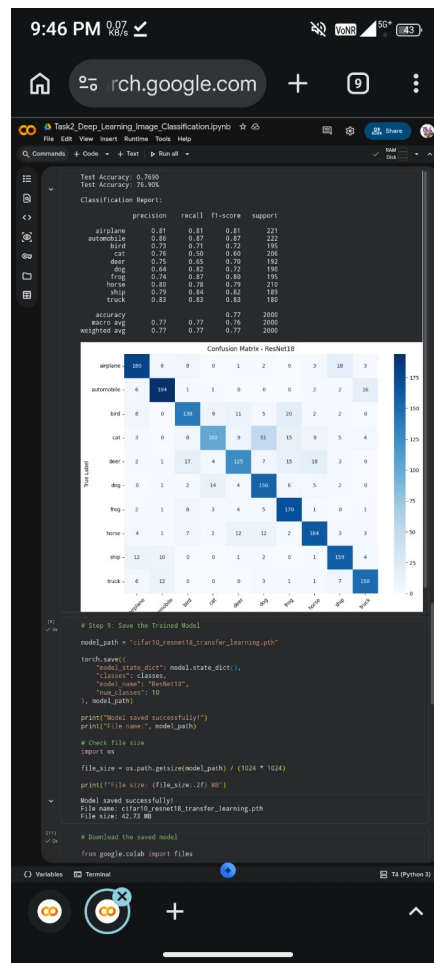
plt.title("Confusion Matrix - ResNet18")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

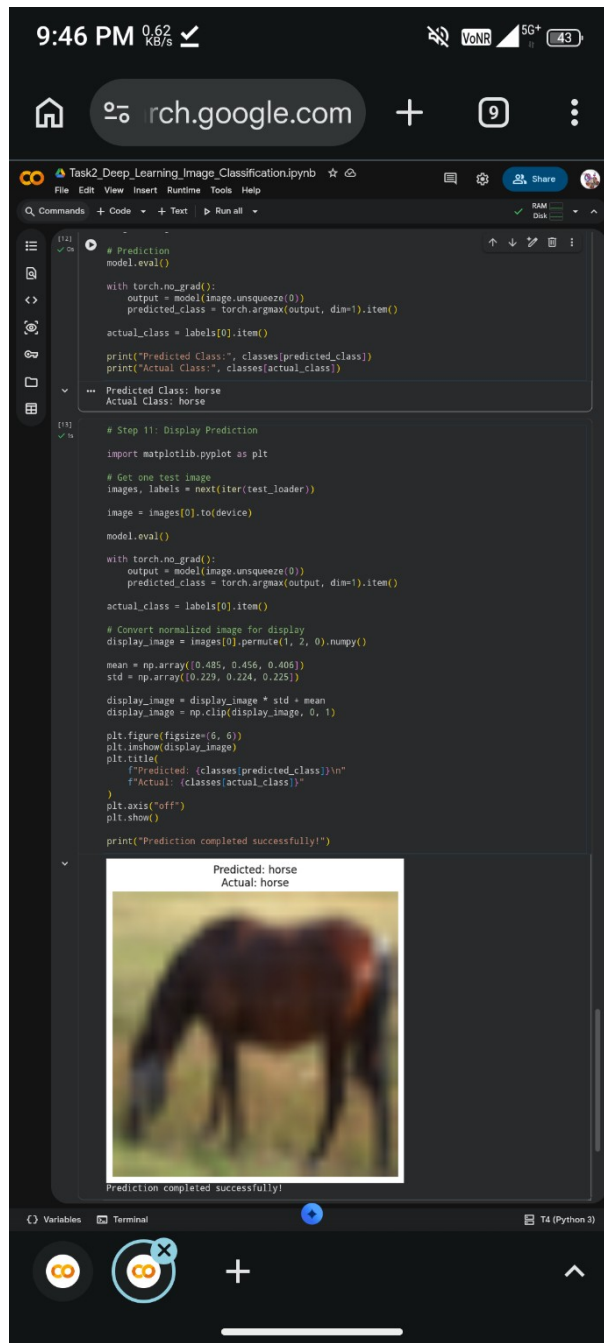
Test Accuracy: 0.7690
Test Accuracy: 76.90%
Classification Report:
```

6. Saved Model and Inference

The trained model was saved as **cifar10_resnet18_transfer_learning.pth**. The checkpoint contains the model state dictionary, class names, model name, and number of classes.

Inference steps: load the model, apply the test transformations, run the image through the model in evaluation mode, and select the predicted class.





7. Conclusion

The transfer-learning based CIFAR-10 image classifier was completed using PyTorch and ResNet18. Training accuracy reached 74.68%, and test accuracy reached 76.90% on the selected subsets.